



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

数据安全实验报告

Lab 2: 零知识证明实验

陈语童

年级：2023 级

学号：2311887

专业：计算机科学与技术

课程教师：刘哲理

2026 年 5 月 2 日

目录

一、 实验要求	2
二、 实验原理	2
(一) 零知识证明	2
(二) Groth16	2
(三) R1CS	3
三、 环境配置	4
(一) Libsnark 安装	4
四、 实验过程	6
(一) 代码编写	6
1. 将待证明的命题表达为 R1CS	6
2. 生成证明密钥和验证密钥	8
3. 证明方使用证明密钥和可行解构造证明	8
4. 验证方使用验证密钥验证证明	10
(二) 编译运行	10
五、 实验总结	15

一、实验要求

参考教材实验 4.1，假设 Alice 希望证明自己知道如下方程的解：

$$x^3 + x + 5 = out$$

其中 out 是大家都知道的一个数，这里假设 $out = 35$ ，而 $x = 3$ 就是方程的解。请实现代码完成证明生成和证明的验证。

二、实验原理

（一）零知识证明

零知识证明是一种涉及两方或更多方的协议，允许证明者能够在不向验证者提供任何有用的信息的情况下，使验证者相信某个论断是正确的。

定义：零知识（Zero-Knowledge, ZK）证明允许证明方让验证方相信证明方自己知道一个满足 $C(x) = 1$ 的 x ，但不会进一步泄漏关于 x 的任何信息。

其中 C 是一个公开的谓词函数，返回 bool。该定义意味着一个零知识证明通常需要将证明过程转化为验证一个谓词函数是否成立的形式。

零知识证明中，需要满足三个性质：

- **正确性：**没有人能够假冒证明方 P 使这个证明成功。如果不满足这条性质，也就是证明方 P 不知道“知识”，再怎么证明，验证方 V 也很难相信证明方 P 拥有正确的知识。
- **完备性：**如果证明方 P 和验证方 V 都是诚实的，并且证明过程的每一步都进行正确的计算，那么这个证明一定是成功的。也就是说如果证明方 P 知道“知识”，那么验证方 V 会有极大的概率相信证明方 P 。
- **零知识性：**证明执行完之后，验证方 V 只获得了“证明方 P 拥有这个知识”这条信息，而没有获得关于这个知识本身的任何信息。

零知识证明应用场景很广，不限于身份认证、数据隐私保护（金融保护、区块链等）等诸多方面。

（二）Groth16

Groth16 是目前最经典、也是实际应用中最广泛的 zkSNARK（零知识简洁非交互式论证）证明系统之一，其优点是证明短、验证快，缺点是需要针对每个电路进行一次可信设置（trusted setup）。

简单来说，它主要用于证明：**我知道某个秘密输入，使得一个计算（电路）成立，但证明时又不能泄露这个秘密。**

Groth16 有 3 个核心步骤：

1. **Setup（初始化/可信设置）：**基于电路（或约束系统），生成证明密钥（Proving Key, PK）给证明者使用；生成验证密钥（Verification Key, VK）给验证者使用。本质上，是把程序转换成数学形式，生成一个随机秘密参数并用它们构造承诺。
2. **Prove（生成证明）：**由证明者完成。根据 PKey、公共输入 x 、私有输入 w （证明者的秘密），证明者需要证明“存在 w ，使得电路 $C(x, w) = 1$ ”。系统会把“这个答案是否正确”

转换成一组多项式关系，证明者就用 w 去填这些数学公式，生成三个椭圆曲线点，在不透露秘密答案或完整计算过程的情况下作为**证明**发出，供验证者验证。

3. **Verify (验证证明)**: 由验证者完成。验证者接收证明者发来的证明，根据 VKey 和公共输入 x ，根据固定的**验证规则**（含有上述变量/元素的数学等式）进行验证，通过验证（等式成立）说明证明者发出的证明是正确/成功的零知识证明。

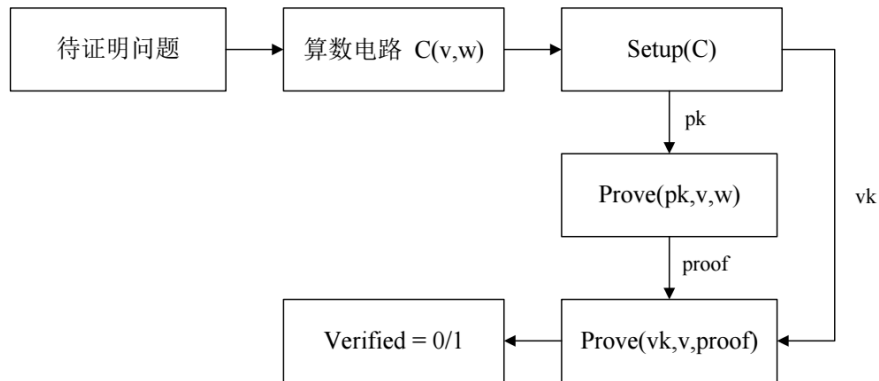


图 1: Groth16 证明系统流程示意图

(三) R1CS

R1CS (Rank-1 Constraint System, 秩 1 约束系统) 把一个计算过程改写成一堆“乘法等式”的形式。它是 Groth16 等 zk-SNARK 的基础表示方式。

R1CS 的任何一条约束形式为：

$$\text{线性组合} \times \text{线性组合} = \text{线性组合}$$

由于每个约束都是“一个线性组合 $\times$ 一个线性组合”，这种结构在线性代数里对应 rank-1 的形式，故叫做 Rank-1 约束系统。

R1CS 的目的是把任意程序(加法、乘法、逻辑等)拆成很多条“乘法约束”，从而使 zk-SNARK 能够进行进一步处理。

例如，证明方有一个整数 x ，他希望向验证方证明这个整数 x 的取值范围为 $[0, 3]$ 。那么在 Groth16 之前需要把命题转化为 R1CS 表示。如果想要约束整数 x 必须取值于 $[a, b]$ ，可以用如下公式来约束它：

$$(x - a)(x - (a + 1)) \cdots (x - b) = 0$$

对于“证明整数 x 的取值范围为 $[0, 3]$ ”这一问题，可以写出如下约束：

$$x(x - 1)(x - 2)(x - 3) = 0$$

用 v 表示公有输入, w 表示证明方需要证明的私密输入值, 将算数电路拍平成多个 $x = y$ 或者 $x = y \text{ op } z$ 形式的等式, 其中 op 可以是加、减、乘、除运算符中的一种。对于 $x(x-1)(x-2)(x-3) = 0$,

可以拍平成如下几个等式：

$$w_1 = x - 1$$

$$w_2 = x - 2$$

$$w_3 = x - 3$$

$$w_4 = x \cdot w_1$$

$$w_5 = w_2 \cdot w_4$$

$$out = w_3 \cdot w_5$$

这就是 R1CS 约束的生成。

三、 环境配置

本实验在 WSL2、Ubuntu 22.04 中进行。

(一) Libsnark 安装

在 [libsnark_abc 仓库](#) 下载源代码 ZIP，解压到 WSL 中新建文件夹 `~/Libsnark` 下，得到 `~/Libsnark/libsnark_abc-master`。

从 [libsnark 仓库](#) 下载源代码 ZIP，解压到 `~/Libsnark/libsnark_abc-master/depends/libsnark` 文件夹内。

从 [libsnark depends 目录](#) 下载 6 个子模块的源代码：

Name	Last commit message	Last commit date
...		
ate-pairing @ e698901	Update git submodules to latest commits on origin	9 years ago
gtest @ 3a4cf1a	Update libqfft, libff, gtest, xbyak dependency versions	8 years ago
libff @ 176f34	Updated libff, libqfft submodules to their latest staging branch	6 years ago
libqfft @ 7e1e957	Updated libff, libqfft submodules to their latest staging branch	6 years ago
libsnark-supercop @ b04a0ea	Update third_party to depends	9 years ago
xbyak @ f0a8f7f	Update libqfft, libff, gtest, xbyak dependency versions	8 years ago
CMakeLists.txt	Adds linked library compilation support for libqfft and libff	8 years ago

图 2: libsnark 子模块下载页面

下载后解压，按照指导手册中的说明更改文件夹名：

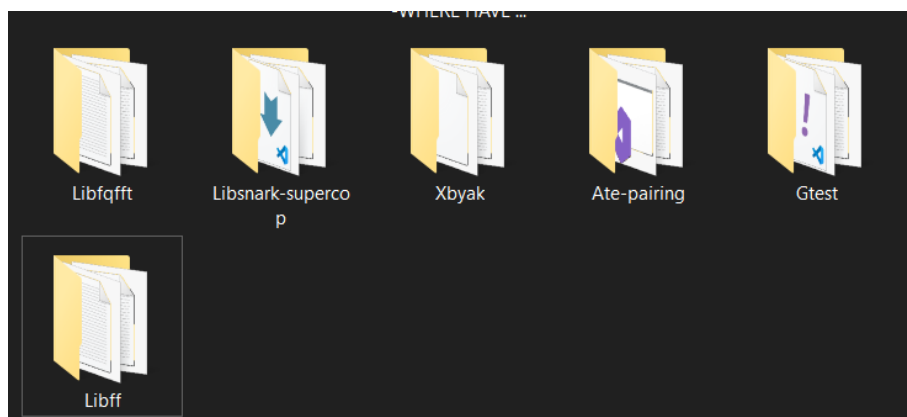


图 3: 子模块文件夹命名要求

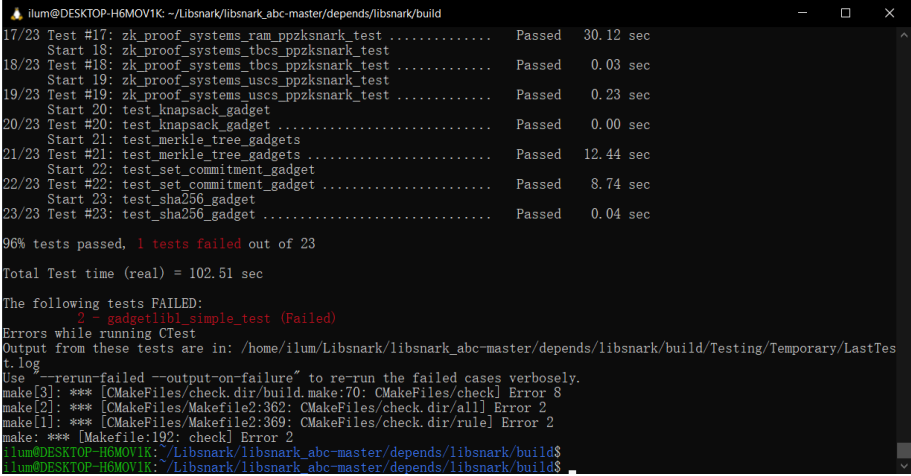
而后，在 Ubuntu 中执行以下命令，安装编译/构建所需的依赖工具和开发库：

```
sudo apt install build-essential cmake git libgmp3-dev libprocps-dev python3-
markdown libboost-program-options-dev libssl-dev python3 pkg-config
```

再将下载的子模块文件依次复制到 Ubuntu 中，用终端命令进行安装。过程比较复杂，涉及一些与 Ubuntu 22.04 不兼容的处理，此处就不一一注解了。

都安装成功后，在 `~/Libsnark/libsnark_abc-master/depends/libsnark` 下打开终端，执行以下命令：

```
mkdir build
cd build
cmake ..
make
make check
```



```
ilum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/depends/libsnark/build
17/23 Test #17: zk_proof_systems_ram_ppzksnark_test ..... Passed 30.12 sec
Start 18: zk_proof_systems_tbc_ppzksnark_test .....
18/23 Test #18: zk_proof_systems_tbc_ppzksnark_test ..... Passed 0.03 sec
Start 19: zk_proof_systems_uscs_ppzksnark_test .....
19/23 Test #19: zk_proof_systems_uscs_ppzksnark_test ..... Passed 0.23 sec
Start 20: test_knapsack_gadget .....
20/23 Test #20: test_knapsack_gadget ..... Passed 0.00 sec
Start 21: test_merkle_tree_gadgets .....
21/23 Test #21: test_merkle_tree_gadgets ..... Passed 12.44 sec
Start 22: test_set_commitment_gadget .....
22/23 Test #22: test_set_commitment_gadget ..... Passed 8.74 sec
Start 23: test_sha256_gadget .....
23/23 Test #23: test_sha256_gadget ..... Passed 0.04 sec

96% tests passed, 1 tests failed out of 23

Total Test time (real) = 102.51 sec

The following tests FAILED:
  2 - gadgetlib1_simple_test (Failed)
Errors while running CTest
Output from these tests are in: /home/ilum/Libsnark/libsnark_abc-master/depends/libsnark/build/Testing/Temporary/LastTest
0.log
Use --rerun-failed --output-on-failure to re-run the failed cases verbosely.
make[3]: *** [CMakeFiles/check.dir/build.make:70: CMakeFiles/check] Error 8
make[2]: *** [CMakeFiles/Makefile2:362: CMakeFiles/check.dir/all] Error 2
make[1]: *** [CMakeFiles/Makefile2:369: CMakeFiles/check.dir/rule] Error 2
make: *** [Makefile:192: check] Error 2

ilum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/depends/libsnark/build$
ilum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/depends/libsnark/build$
```

图 4: libsnark 依赖测试结果

测试结果如上。有一个测试不通过，可能是密码学库太老和新的编译器有不匹配、不兼容导致的。实际项目中可能已经是可以使用的状态，暂且可以不管此测试的错误。

所有依赖安装并编译成功后，在 `~/Libsnark/libsnark_abc-master` 下打开终端，执行以下命令：

```
mkdir build
cd build
cmake ..
make
```

编译完成后，执行：

```
./src/test
```

出现以下日志：

```

ilium@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build
(leave) Call to alt_bn128_final_exponentiation_first_chunk [0.0001s x1.01] (1777644167.3720s x0.00 from sta^
rt)
(enter) Call to alt_bn128_final_exponentiation_last_chunk [ ] (1777644167.3721s x0.00 from sta
rt)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777644167.3721s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0005s x1.01] (1777644167.3726s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777644167.3726s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0005s x1.01] (1777644167.3731s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777644167.3731s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0005s x1.01] (1777644167.3736s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation_last_chunk [0.0017s x1.01] (1777644167.3737s x0.00 from sta
rt)
(leave) Call to alt_bn128_final_exponentiation [0.0018s x1.01] (1777644167.3738s x0.00 from start)
(leave) Check QAP divisibility [0.0048s x1.01] (1777644167.3738s x0.00 from start)
(leave) Online pairing computations [0.0049s x1.01] (1777644167.3738s x0.00 from start)
(leave) Call to rlcs_gg_ppzksnark_online_verifier_weak_IC [0.0050s x1.01] (1777644167.3738s x0.00 from start)
(leave) Call to rlcs_gg_ppzksnark_online_verifier_strong_IC [0.0051s x1.01] (1777644167.3739s x0.00 from start)
(leave) Call to rlcs_gg_ppzksnark_verifier_strong_IC [0.0059s x1.01] (1777644167.3739s x0.00 from start)
Number of RICS constraints: 4
Primary (public) input: 1
35
Auxiliary (private) input: 4
3
9
27
30
Verification status: 1
ilium@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build$

```

图 5: zk-SNARKs demo 运行结果

则说明已顺利拥有了 zkSNARK 应用开发环境，并成功跑了第一个 zk-SNARKs demo。

四、 实验过程

(一) 代码编写

1. 将待证明的命题表达为 R1CS

将算数电路拍平成多个 $x = y$ 或者 $x = y \text{ op } z$ 形式的等式，其中 op 可以是加、减、乘、除运算符中的一种。对于 $x^3 + x + 5 = out$ ，可以拍平成如下几个等式：

$$\begin{aligned}
 w_1 &= x \cdot x \\
 w_2 &= w_1 \cdot x \\
 w_3 &= w_2 + x \\
 out &= w_3 + 5
 \end{aligned}$$

其中公开输入 $out = 35$ ， w_n 为私密输入，只有证明方才会知道。

而后构建一个 protoboard，用来把所有变量、组件和约束关联起来。参考手册中的 common.hpp，将其中涉及到 R1CS 的部分替换成本实验命题对应的约束系统。也就是用代码形式表达出上面拍平的等式。替换后代码如下：

```

1 #include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
2 #include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
3 #include <libsark/gadgetlib1/pb_variable.hpp>
4
5 using namespace libsark;
6 using namespace std;
7

```

```

8 //定义使用的有限域
9 typedef libff::Fr<default_r1cs_gg_ppzksnark_pp> FieldT;
10
11 //定义创建面包板的函数
12 protoboard<FieldT> build_protoboard(int* secret)
13 {
14     //初始化曲线参数
15     default_r1cs_gg_ppzksnark_pp::init_public_params();
16     //创建面包板
17     protoboard<FieldT> pb;
18     //定义所有需要外部输入的变量以及中间变量（已替换）
19     pb_variable<FieldT> x;
20     pb_variable<FieldT> w_1;
21     pb_variable<FieldT> w_2;
22     pb_variable<FieldT> w_3;
23     pb_variable<FieldT> out;
24     // 将各个变量与protoboard连接，把各个元器件插到“面包板”上（已替换）
25     out.allocate(pb, "out");
26     x.allocate(pb, "x");
27     w_1.allocate(pb, "w_1");
28     w_2.allocate(pb, "w_2");
29     w_3.allocate(pb, "w_3");
30     //定义公有的变量的数量，set_input_sizes(n)用来声明与protoboard连接的public变量的个数n
31     pb.set_input_sizes(1);
32     //为公有变量赋值（已替换 out = 35）
33     pb.val(out) = 35;
34
35     /* 至此，所有变量都已经顺利与protoboard相连，
36      * 下面需要确定的是这些变量间的约束关系。
37      * 调用protoboard 的add_r1cs_constraint()函数，
38      * 为pb添加各种运算的r1cs_constraint。
39      */
40     // x * x = w_1
41     pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, x, w_1));
42
43     // w_1 * x = w_2
44     pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_1, x, w_2));
45
46     // w_2 + x = w_3
47     pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_2 + x, 1, w_3));
48
49     // w_3 + 5 = out
50     pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_3 + 5, 1, out));
51
52     //证明者在生成证明阶段传入私密输入，为私密变量赋值，其他阶段为NULL
53     if (secret != NULL)
54     {
55         pb.val(x) = secret[0];

```



```

56     pb.val(w_1) = secret[1];
57     pb.val(w_2) = secret[2];
58     pb.val(w_3) = secret[3];
59 }
60
61 return pb;
62 }

```

2. 生成证明密钥和验证密钥

针对命题的电路构建完毕，接下来是生成公钥的初始设置阶段（Trusted Setup）。

这一阶段使用生成算法为该命题生成公共参数（证明密钥和验证密钥，证明密钥供证明者使用，验证密钥供验证者使用），并把生成的密钥输出到对应文件保存。

由于整体 project 的结构和对应面包板的 interface 都没有改变，保持手册中参考代码的实现即可：

```

1  #include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
2  #include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
3  #include <fstream>
4  #include "common.hpp"
5
6  using namespace libsark;
7  using namespace std;
8
9  int main()
10 {
11     //构造面包板
12     protoboard<FieldT> pb = build_protoboard(NULL);
13     const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();
14     //生成证明密钥和验证密钥
15     const r1cs_gg_ppzksnark_keypair<default_r1cs_gg_ppzksnark_pp> keypair =
16         r1cs_gg_ppzksnark_generator<default_r1cs_gg_ppzksnark_pp>(constraint_system);
17     //保存证明密钥到文件pk.raw
18     fstream pk("pk.raw", ios_base::out);
19     pk << keypair.pk;
20     pk.close();
21     //保存验证密钥到文件vk.raw
22     fstream vk("vk.raw", ios_base::out);
23     vk << keypair.vk;
24     vk.close();
25
26     return 0;
27 }

```

3. 证明方使用证明密钥和可行解构造证明

在构造证明阶段，证明者需要为 private input (w_n) 提供具体数值，再把已经公开的 public input (out) 以及 private input 的数值传给 prover 函数生成证明，程序运行生成的证明会保存

到 `proof.raw` 文件供验证者使用。

参考手册中的 `myprove.cpp`，将其中 R1CS 表达式相关的代码进行替换即可：

```
1 #include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
2 #include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
3 #include <fstream>
4 #include "common.hpp"
5
6 using namespace libsark;
7 using namespace std;
8
9 int main()
10 {
11     //输入秘密值x
12     int x;
13     cin >> x;
14     //为私密输入提供具体数值（已替换为本实验R1CS）
15     int secret[4];
16     secret[0] = x;
17     secret[1] = x * x;
18     secret[2] = x * x * x;
19     secret[3] = x * x * x + x;
20     //构造面包板
21     protoboard<FieldT> pb = build_protoboard(secret);
22     const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();
23     cout << "Number of R1CS constraints: " << constraint_system.num_constraints() <<
24         endl;
25     cout << "Public input: " << pb.primary_input() << endl;
26     cout << "Private input: " << pb.auxiliary_input() << endl;
27     //加载证明密钥
28     fstream f_pk("pk.raw", ios_base::in);
29     r1cs_gg_ppzksnark_proving_key<default_r1cs_gg_ppzksnark_pp> pk;
30     f_pk >> pk;
31     f_pk.close();
32     //生成证明
33     const r1cs_gg_ppzksnark_proof<default_r1cs_gg_ppzksnark_pp> proof =
34         r1cs_gg_ppzksnark_prover<default_r1cs_gg_ppzksnark_pp>(pk, pb.primary_input(),
35             pb.auxiliary_input());
36     //将生成的证明保存到proof.raw文件
37     fstream pr("proof.raw", ios_base::out);
38     pr << proof;
39     pr.close();
40     return 0;
41 }
```

4. 验证方使用验证密钥验证证明

验证方使用 `verifier` 函数校验证明。若验证结果 `verified = 1`，说明验证成功/正确。同样的，由于整体架构和接口都没有改变，使用指导教材给的原代码就可以：

```

1 #include <libsark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
2 #include <libsark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
3 #include <fstream>
4 #include "common.hpp"
5
6 using namespace libsark;
7 using namespace std;
8
9 int main()
10 {
11     //构造面包板
12     protoboard<FieldT> pb = build_protoboard(NULL);
13     const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();
14     //加载验证密钥
15     fstream f_vk("vk.raw", ios_base::in);
16     r1cs_gg_ppzksnark_verification_key<default_r1cs_gg_ppzksnark_pp> vk;
17     f_vk >> vk;
18     f_vk.close();
19     //加载银行生成的证明
20     fstream f_proof("proof.raw", ios_base::in);
21     r1cs_gg_ppzksnark_proof<default_r1cs_gg_ppzksnark_pp> proof;
22     f_proof >> proof;
23     f_proof.close();
24     //进行验证
25     bool verified =
26         r1cs_gg_ppzksnark_verifier_strong_IC<default_r1cs_gg_ppzksnark_pp>(vk, pb.
27         primary_input(), proof);
28
29     cout << "Number of R1CS constraints: " << constraint_system.num_constraints() <<
30     endl;
31     cout << "Public input: " << pb.primary_input() << endl;
32     cout << "Verification result: " << verified << endl;
33
34     return 0;
35 }

```

(二) 编译运行

实验代码文件直接创建在 `~/Libsark/libsark_abc-master/src` 目录下。本次写入的文件包括 `common.hpp`、`mysetup.cpp`、`myprove.cpp`、`myverify.cpp`。因此，不再需要像指导教材中一样用 `touch` 命令去新建文件并写入。

但仍然需要修改 `CMakeLists.txt`。打开 `~/Libsark/libsark_abc-master/src` 目录下的 `CMakeLists.txt` 文件，将如下代码复制到文件末尾：

```
add_executable(
    mysetup
    mysetup.cpp
)
target_link_libraries(
    mysetup
    snark
)
target_include_directories(
    mysetup
    PUBLIC
    ${DEPENDS_DIR}/libsnaek
    ${DEPENDS_DIR}/libsnaek/depends/libfqfft
)

add_executable(
    myprove
    myprove.cpp
)
target_link_libraries(
    myprove
    snark
)
target_include_directories(
    myprove
    PUBLIC
    ${DEPENDS_DIR}/libsnaek
    ${DEPENDS_DIR}/libsnaek/depends/libfqfft
)

add_executable(
    myverify
    myverify.cpp
)
target_link_libraries(
    myverify
    snark
)
target_include_directories(
    myverify
    PUBLIC
    ${DEPENDS_DIR}/libsnaek
    ${DEPENDS_DIR}/libsnaek/depends/libfqfft
)
```

在 ~/Libsnark/libsnark_abc-master/build 下打开终端，执行以下命令进行编译，并进

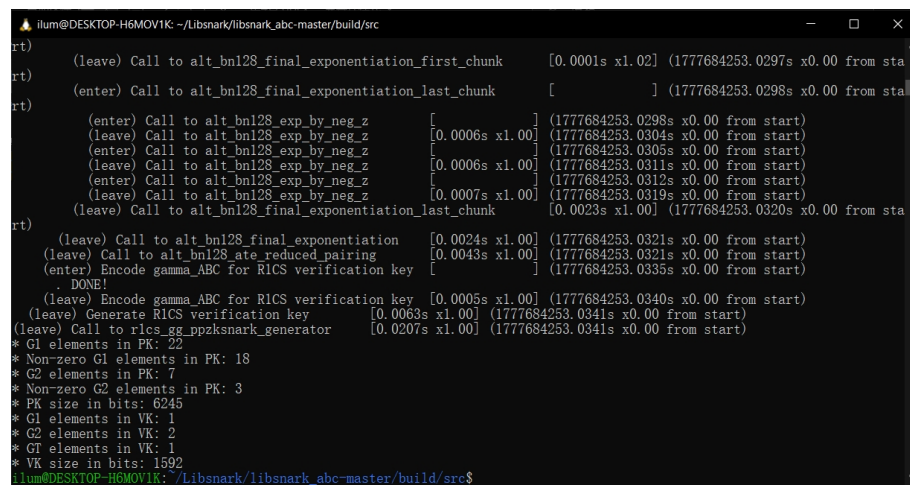
入生成的可执行文件目录：

```
cmake ..
make
cd src
```

而后依次运行：

```
./mysetup
```

进行公钥的初始设置：



```

rt) (leave) Call to alt_bn128_final_exponentiation_first_chunk [0.0001s x1.02] (1777684253.0297s x0.00 from sta
rt) (enter) Call to alt_bn128_final_exponentiation_last_chunk [ ] (1777684253.0298s x0.00 from sta
rt) (enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684253.0298s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0006s x1.00] (1777684253.0304s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684253.0305s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0006s x1.00] (1777684253.0311s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684253.0312s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0007s x1.00] (1777684253.0319s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation_last_chunk [0.0023s x1.00] (1777684253.0320s x0.00 from sta
rt) (leave) Call to alt_bn128_final_exponentiation [0.0024s x1.00] (1777684253.0321s x0.00 from start)
(leave) Call to alt_bn128_ate_reduced_pairing [0.0043s x1.00] (1777684253.0321s x0.00 from start)
(enter) Encode gamma_ABC for RICS verification key [ ] (1777684253.0335s x0.00 from start)
. DONE!
(leave) Encode gamma_ABC for RICS verification key [0.0005s x1.00] (1777684253.0340s x0.00 from start)
(leave) Generate RICS verification key [0.0063s x1.00] (1777684253.0341s x0.00 from start)
(leave) Call to rics_gg_ppzksnark_generator [0.0207s x1.00] (1777684253.0341s x0.00 from start)
* G1 elements in PK: 22
* G2 elements in PK: 7
* Non-zero G1 elements in PK: 18
* Non-zero G2 elements in PK: 3
* PK size in bits: 6245
* G1 elements in VK: 1
* G2 elements in VK: 2
* GT elements in VK: 1
* VK size in bits: 1592
ilum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src$

```

图 6: 执行 mysetup 后生成证明密钥与验证密钥

以上输出说明证明密钥和验证密钥均已成功生成，并输出到了相应文件中。

```
./myprove
3
```

```

illum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src
(leave) Call to alt_bn128_final_exponentiation_first_chunk [0.0001s x1.02] (1777684253.0297s x0.00 from sta^
rt)
(enter) Call to alt_bn128_final_exponentiation_last_chunk [ ] (1777684253.0298s x0.00 from sta
rt)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684253.0298s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0006s x1.00] (1777684253.0304s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684253.0305s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0006s x1.00] (1777684253.0311s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684253.0312s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0007s x1.00] (1777684253.0319s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation_last_chunk [0.0023s x1.00] (1777684253.0320s x0.00 from sta
rt)
(leave) Call to alt_bn128_final_exponentiation [0.0024s x1.00] (1777684253.0321s x0.00 from start)
(leave) Call to alt_bn128_ate_reduced_pairing [0.0043s x1.00] (1777684253.0321s x0.00 from start)
(enter) Encode gamma_ABC for RICS verification key [ ] (1777684253.0335s x0.00 from start)
. DONE!
(leave) Encode gamma_ABC for RICS verification key [0.0005s x1.00] (1777684253.0340s x0.00 from start)
(leave) Generate RICS verification key [0.0063s x1.00] (1777684253.0341s x0.00 from start)
(leave) Call to rics_gg_ppzksnark_generator [0.0207s x1.00] (1777684253.0341s x0.00 from start)
* G1 elements in PK: 22
* Non-zero G1 elements in PK: 18
* G2 elements in PK: 7
* Non-zero G2 elements in PK: 3
* PK size in bits: 6245
* G1 elements in VK: 1
* G2 elements in VK: 2
* G1 elements in VK: 1
* VK size in bits: 1592
illum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src$ ./myprove

```

图 7: 执行 myprove 并输入 $x = 3$

运行后会要求提供验证用的具体数值。根据实验题目，证明方要证明的是 $x = 3$ 是方程的解，所以在 console 输入 3 即可。

```

illum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src
(enter) Compute the proof [ ] (1777684285.6959s x0.00 from start)
(enter) Compute evaluation to A-query [ ] (1777684285.6959s x0.00 from start)
(enter) Process scalar vector [ ] (1777684285.6960s x0.00 from start)
* Elements of w skipped: 0 (0.00%)
* Elements of w processed with special addition: 1 (16.67%)
(leave) Process scalar vector [0.0000s x1.03] (1777684285.6960s x0.00 from start)
(leave) Compute evaluation to A-query [0.0001s x1.01] (1777684285.6961s x0.00 from start)
(enter) Compute evaluation to B-query [ ] (1777684285.6961s x0.00 from start)
(enter) Process scalar vector [ ] (1777684285.6961s x0.00 from start)
* Elements of w skipped: 0 (0.00%)
* Elements of w processed with special addition: 1 (50.00%)
* Elements of w remaining: 1 (50.00%)
(leave) Process scalar vector [0.0001s x1.01] (1777684285.6962s x0.00 from start)
(leave) Compute evaluation to B-query [0.0002s x1.00] (1777684285.6963s x0.00 from start)
(enter) Compute evaluation to H-query [ ] (1777684285.6963s x0.00 from start)
(leave) Compute evaluation to H-query [0.0012s x1.00] (1777684285.6975s x0.00 from start)
(enter) Compute evaluation to L-query [ ] (1777684285.6976s x0.00 from start)
(enter) Process scalar vector [ ] (1777684285.6976s x0.00 from start)
* Elements of w skipped: 0 (0.00%)
* Elements of w processed with special addition: 0 (0.00%)
* Elements of w remaining: 4 (100.00%)
(leave) Process scalar vector [0.0000s x1.04] (1777684285.6976s x0.00 from start)
(leave) Compute evaluation to L-query [0.0001s x0.99] (1777684285.6976s x0.00 from start)
(leave) Compute the proof [0.0050s x1.00] (1777684285.7009s x0.00 from start)
(leave) Call to rics_gg_ppzksnark_prover [0.0061s x1.00] (1777684285.7009s x0.00 from start)
* G1 elements in proof: 2
* G2 elements in proof: 1
* Proof size in bits: 1019
illum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src$

```

图 8: 成功生成证明文件

以上输出说明成功生成了证明文件。

```
./myverify
```

最后运行验证程序，它会对证明方发出的证明进行验证，验证成功则输出 `verified = 1`，否则输出 `verified = 0`。

```

illum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src
(leave) Call to alt_bn128_ate_miller_loop [0.0017s x1.00] (1777684293.9402s x0.00 from start)
(enter) Call to alt_bn128_ate_double_miller_loop [ ] (1777684293.9403s x0.00 from start)
(leave) Call to alt_bn128_ate_double_miller_loop [0.0027s x1.00] (1777684293.9430s x0.00 from start)
(enter) Call to alt_bn128_final_exponentiation [ ] (1777684293.9430s x0.00 from start)
(enter) Call to alt_bn128_final_exponentiation_first_chunk [ ] (1777684293.9430s x0.00 from sta
rt)
(leave) Call to alt_bn128_final_exponentiation_first_chunk [0.0001s x1.01] (1777684293.9431s x0.00 from sta
rt)
(enter) Call to alt_bn128_final_exponentiation_last_chunk [ ] (1777684293.9431s x0.00 from sta
rt)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684293.9432s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684293.9441s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684293.9442s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684293.9450s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684293.9451s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684293.9460s x0.00 from start)
(enter) Call to alt_bn128_final_exponentiation_last_chunk [0.0030s x1.00] (1777684293.9462s x0.00 from sta
rt)
(leave) Call to alt_bn128_final_exponentiation [0.0032s x1.00] (1777684293.9462s x0.00 from start)
(leave) Check QAP divisibility [0.0085s x1.00] (1777684293.9462s x0.00 from start)
(leave) Online pairing computations [0.0085s x1.00] (1777684293.9462s x0.00 from start)
(leave) Call to rlcs_gg_ppzksnark_online_verifier_weak_IC [0.0086s x1.00] (1777684293.9463s x0.00 from start)
(leave) Call to rlcs_gg_ppzksnark_online_verifier_strong_IC [0.0087s x1.00] (1777684293.9463s x0.00 from start)
(leave) Call to rlcs_gg_ppzksnark_verifier_strong_IC [0.0102s x1.00] (1777684293.9463s x0.00 from start)
Number of RICS constraints: 4
Public input: 1
35
Verification result: 1
illum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src$

```

图 9: 验证正确解 $x = 3$ 的证明

输出 Verification result: 1, 即 verified = 1, 说明成功验证了该证明。

可以进一步测试, 如果发出证明时输入的不是 $x = 3$, 而是 $x = 1$:

```

illum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src
rt) (leave) Call to alt_bn128_final_exponentiation_first_chunk [0.0001s x1.00] (1777684300.8778s x0.00 from sta
rt)
rt) (enter) Call to alt_bn128_final_exponentiation_last_chunk [ ] (1777684300.8779s x0.00 from sta
rt)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684300.8779s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684300.8788s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684300.8789s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0008s x1.00] (1777684300.8796s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [ ] (1777684300.8797s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0007s x1.00] (1777684300.8804s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation_last_chunk [0.0027s x1.00] (1777684300.8805s x0.00 from sta
rt)
(leave) Call to alt_bn128_final_exponentiation [0.0029s x1.00] (1777684300.8806s x0.00 from start)
(leave) Call to alt_bn128_ate_reduced_pairing [0.0053s x1.00] (1777684300.8806s x0.00 from start)
(enter) Encode gamma_ABC for RICS verification key [ ] (1777684300.8822s x0.00 from start)
. DONE!
(leave) Encode gamma_ABC for RICS verification key [0.0006s x1.00] (1777684300.8828s x0.00 from start)
(leave) Generate RICS verification key [0.0076s x1.00] (1777684300.8828s x0.00 from start)
(leave) Call to rlcs_gg_ppzksnark_generator [0.0239s x1.00] (1777684300.8829s x0.00 from start)
* G1 elements in PK: 22
* Non-zero G1 elements in PK: 18
* G2 elements in PK: 7
* Non-zero G2 elements in PK: 3
* PK size in bits: 6245
* G1 elements in VK: 1
* G2 elements in VK: 2
* GT elements in VK: 1
* VK size in bits: 1592
illum@DESKTOP-H6MOV1K: ~/Libsnark/libsnark_abc-master/build/src$ ./myprove

```

图 10: 执行 myprove 并输入错误解 $x = 1$

最终验证输出:

```

ilum@DESKTOP-H6MOV1K: ~/libsark/libsark-abc-master/build/src
(enter) Call to alt_bn128_ate_double_miller_loop [0.0026s x1.00] (1777684317.7487s x0.00 from start)
(leave) Call to alt_bn128_ate_double_miller_loop [0.0026s x1.00] (1777684317.7513s x0.00 from start)
(enter) Call to alt_bn128_final_exponentiation [0.0001s x1.02] (1777684317.7514s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation [0.0001s x1.02] (1777684317.7515s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684317.7516s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684317.7524s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684317.7525s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684317.7534s x0.00 from start)
(enter) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684317.7534s x0.00 from start)
(leave) Call to alt_bn128_exp_by_neg_z [0.0009s x1.00] (1777684317.7543s x0.00 from start)
(enter) Call to alt_bn128_final_exponentiation_last_chunk [0.0029s x1.00] (1777684317.7545s x0.00 from start)
(leave) Call to alt_bn128_final_exponentiation [0.0031s x1.00] (1777684317.7546s x0.00 from start)
QAP divisibility check failed.
(leave) Check QAP divisibility [0.0084s x1.00] (1777684317.7546s x0.00 from start)
(leave) Online pairing computations [0.0084s x1.00] (1777684317.7546s x0.00 from start)
(leave) Call to r1cs_gg_ppzksnark_online_verifier_weak_IC [0.0086s x1.00] (1777684317.7546s x0.00 from start)
(leave) Call to r1cs_gg_ppzksnark_online_verifier_strong_IC [0.0086s x1.00] (1777684317.7546s x0.00 from start)
(leave) Call to r1cs_gg_ppzksnark_verifier_strong_IC [0.0102s x1.00] (1777684317.7546s x0.00 from start)
Number of R1CS constraints: 4
Public input: 1
35
Verification result: 0
ilum@DESKTOP-H6MOV1K: ~/libsark/libsark-abc-master/build/src$

```

图 11: 验证错误解 $x = 1$ 的证明失败

输出 Verification result: 0, 即 verified = 0, 说明验证失败, 证明不了上面输入的 $x = 1$ 是解。

五、实验总结

实验通过 C 程序在 libsark 框架中的运行, 通过 Groth16 计算的三步骤 Setup、Prove 和 Verify 模拟了零知识证明的过程。代码实现过程让我对于零知识证明的必要流程有了实际认识, 并且第一次学习到 R1CS 拍平算数电路生成算数式组的方法, 很实用。在环境配置过程中, 还对于一些老的密码库与最新系统不兼容等问题的处理积累了新的经验。最终在 libsark 框架下成功模拟了正确的零知识证明流程。

本次实验的代码已经上传至 GitHub 仓库: [data-security/lab2](https://github.com/data-security/lab2).